

📊 Linear Regression — Complete Guide

Goal: This guide explains Linear Regression from the ground up to advanced topics, in simple language, with graphs and code.

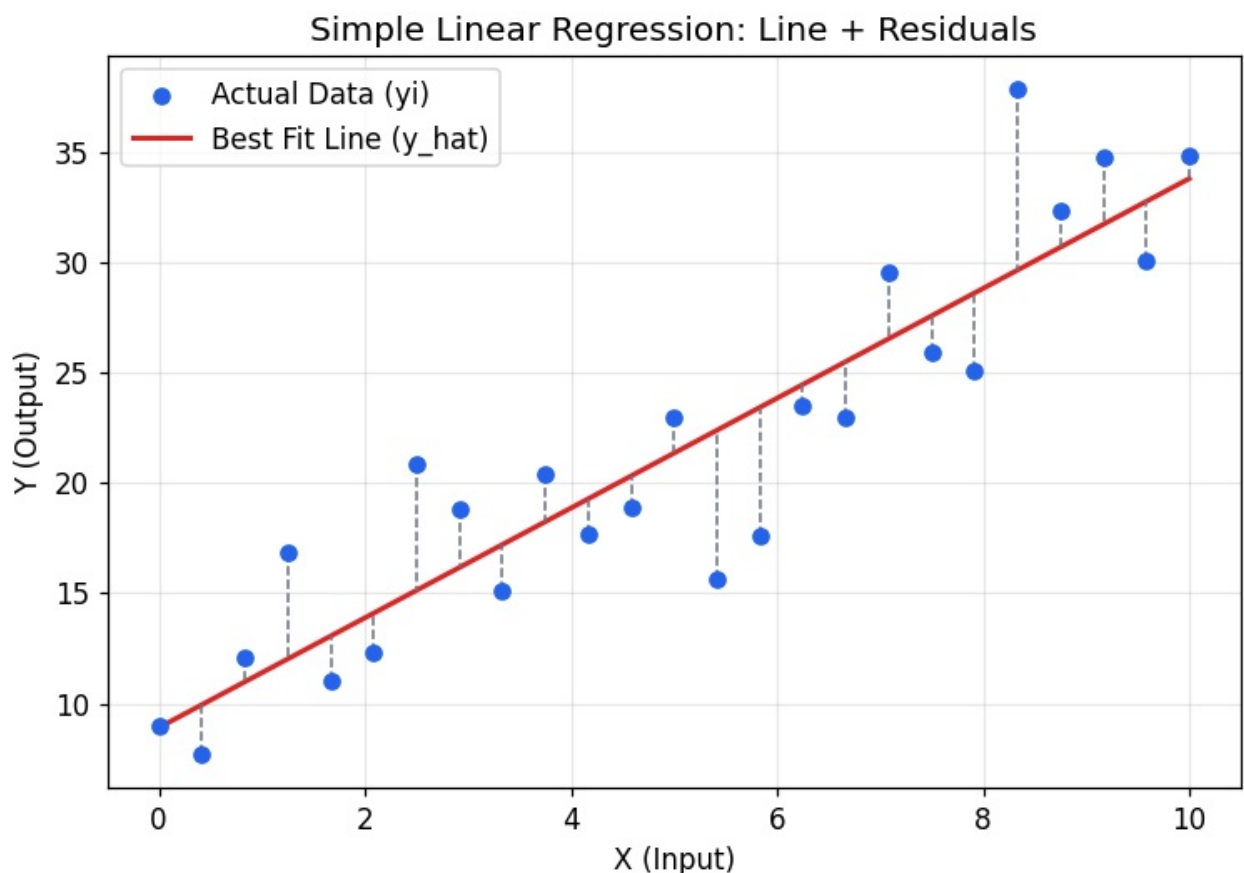
1 Simple Linear Regression (Single Input)

When we have only **one input (X)** and one **output (Y)**, we fit a straight line that describes the relationship between X and Y.

$$\hat{y} = m \cdot x + b$$

$\hat{y}$ = predicted output | m = slope (how steeply the line rises/falls) | b = intercept (where the line crosses the Y-axis)

Example: X = house size (sq ft), Y = house price. The bigger the size, the higher the price — this relationship is modeled with a line.



Blue dots = actual data, red line = best fit line, dashed lines = residuals (errors)

2 Multiple Linear Regression (Multiple Inputs)

When there is more than one input (X1, X2, X3...), we combine them to predict the output. The formula simply extends:

$$\hat{y} = b + m_1 \cdot X_1 + m_2 \cdot X_2 + m_3 \cdot X_3 + \dots + m_n \cdot X_n$$

Example: To predict house price, inputs could be: size, bedrooms, location score, age of house — together they predict the price. Each input has its own weight (m) showing how much it affects the output.

Code (Python — sklearn):

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Multi input example
X = np.array([[1200, 3, 8], [1500, 4, 6], [800, 2, 9], [2000, 5, 5]]) # size, bedrooms, loc
y = np.array([250000, 310000, 180000, 400000]) # price

model = LinearRegression()
model.fit(X, y)

print("Weights (m):", model.coef_)
print("Intercept (b):", model.intercept_)
print("Prediction:", model.predict([[1300, 3, 7]]))
```

3 Residual (Error)

Residual means: **the difference between the actual value and the predicted value**. It shows how wrong the model is at a particular point.

$$\text{residual } (e_i) = y_i - \hat{y}_i$$

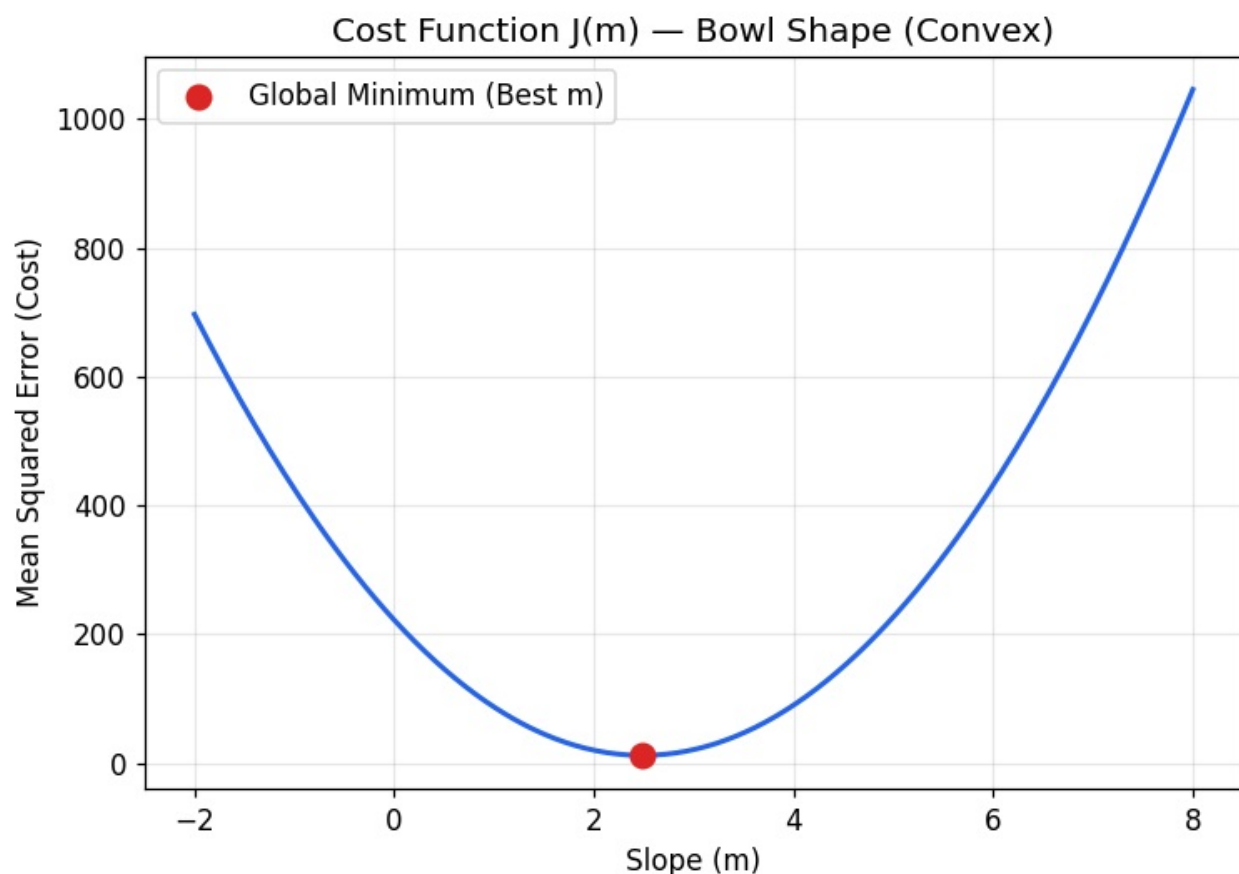
If the residual is close to zero, the prediction is good. If it's large (positive or negative), the model isn't predicting that point well.

4 Mean Squared Error (MSE) — Cost Function

We square all the residuals and average them — this is called the **Cost Function J**. We square them so negative and positive errors don't cancel out, and larger errors get penalized more.

$$\text{MSE} = J(m,b) = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$$

The model's entire goal is to **minimize this Cost Function** — the lower the MSE, the better the model.



The cost function is "bowl"-shaped (U-shape) — the lowest point = the best parameters

5 Gradient Descent

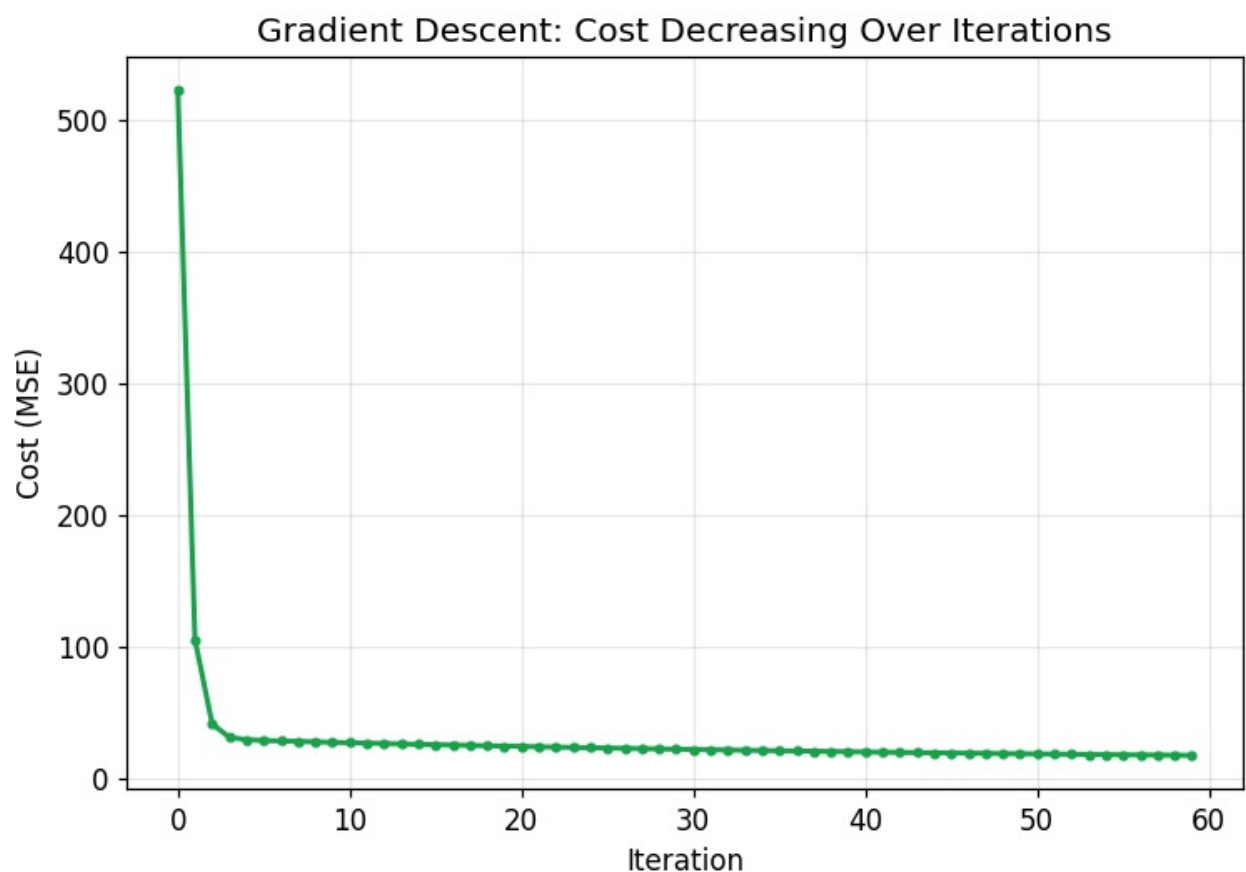
Gradient Descent is an algorithm that takes **small steps** toward the minimum point of the Cost Function (the bottom of the bowl), updating the slope (m) and intercept (b) at every step.

$$m = m - \alpha \cdot \frac{\partial J}{\partial m}$$
$$b = b - \alpha \cdot \frac{\partial J}{\partial b}$$

α (alpha) = Learning Rate — decides how big each step is

The derivatives (gradients) are computed with this formula:

$$\frac{\partial J}{\partial m} = -(2/n) \cdot \sum x_i(y_i - \hat{y}_i)$$
$$\frac{\partial J}{\partial b} = -(2/n) \cdot \sum (y_i - \hat{y}_i)$$



With each iteration, the cost decreases — until it reaches the minimum

Code (Gradient Descent from scratch):

```
import numpy as np

def gradient_descent(x, y, lr=0.01, iterations=1000):
    m, b = 0, 0
    n = len(x)
    for i in range(iterations):
        y_pred = m*x + b
        dm = (-2/n) * sum(x*(y - y_pred))
        db = (-2/n) * sum(y - y_pred)
        m = m - lr*dm
        b = b - lr*db
    return m, b

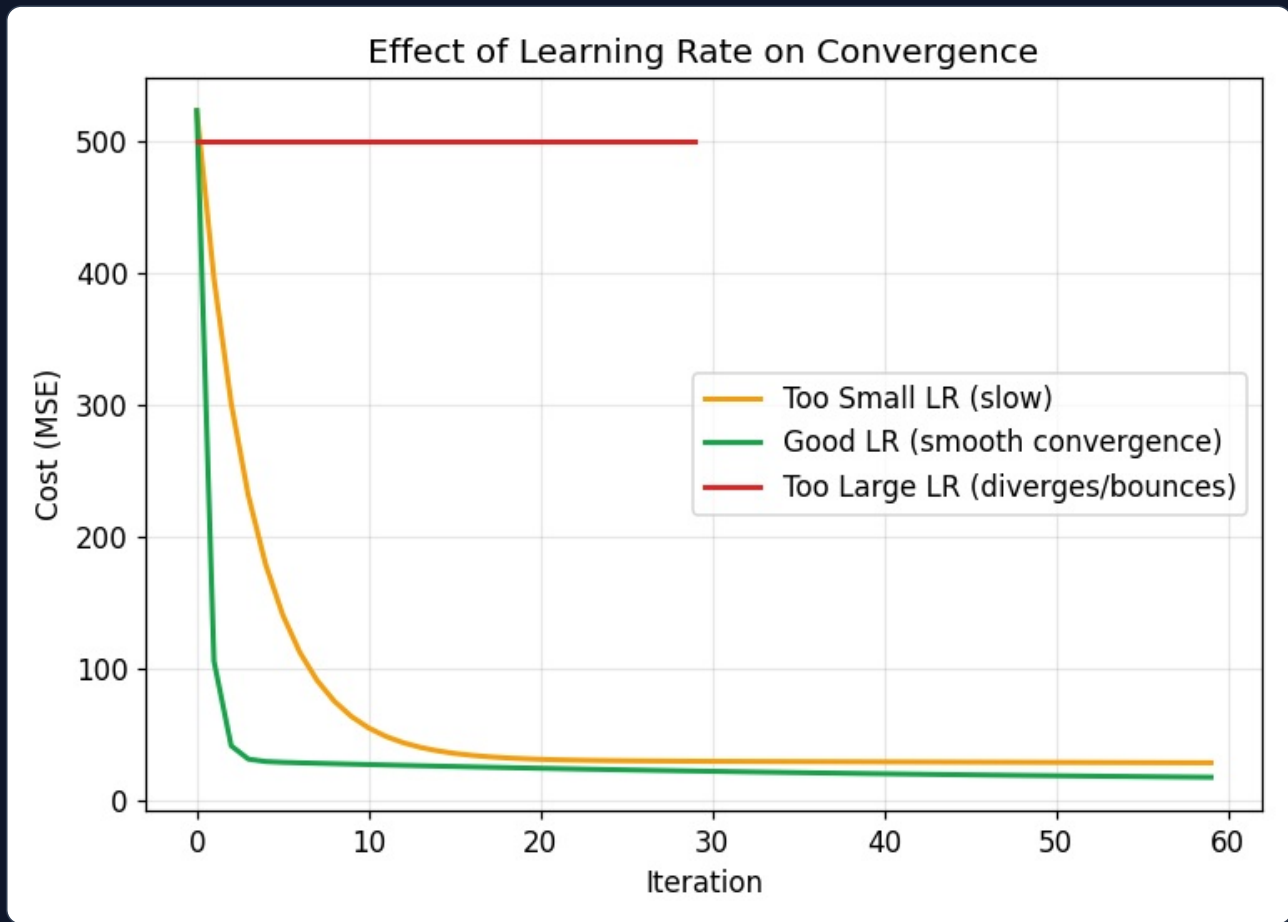
m, b = gradient_descent(np.array([1,2,3,4]), np.array([2,4,6,8]), lr=0.01, iterations=1000)
print(f"m={m:.2f}, b={b:.2f}")
```

6 Learning Rate (α)

The learning rate decides how fast or slow gradient descent learns:

Learning Rate	Effect
Too small (0.0001)	Training is very slow, takes a long time

Just right (0.01 - 0.1 range, needs tuning)	Smooth and fast convergence
Too large (1, 10)	Model "overshoots" the minimum, never converges (diverges)



7 Train-Test Split

We don't train and test the model on the same data, otherwise we won't know how the model will perform on new (unseen) data. So we split the data into two parts:

- **Training set** (usually 70-80%) — the model learns from this
- **Testing set** (usually 20-30%) — we check the model's performance here

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

8 Model Evaluation Metrics

Metric	Formula	What it tells you
MAE (Mean Absolute		

Error)	$(1/n)\sum y_i - \hat{y}_i $	Average error (without squaring)
MSE (Mean Squared Error)	$(1/n)\sum (y_i - \hat{y}_i)^2$	Average squared error, penalizes large errors more
RMSE	$\sqrt{\text{MSE}}$	Square root of MSE — same units as Y
R ² Score	$1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$	How much % variance the model explains (0 to 1, higher is better)

R² (R-Squared)

$$R^2 = 1 - (\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2)$$

R² = 1 means a perfect model. R² = 0 means the model is just predicting the average, it hasn't learned anything.

Adjusted R²

The problem is that R² keeps increasing as we add more features — whether those features are useful or not. So we use **Adjusted R²**, which penalizes useless features.

$$\text{Adjusted } R^2 = 1 - [(1 - R^2)(n - 1) / (n - k - 1)]$$

n = total samples | k = number of input features

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

n, k = X_test.shape[0], X_test.shape[1]
adj_r2 = 1 - (1 - r2) * (n - 1) / (n - k - 1)

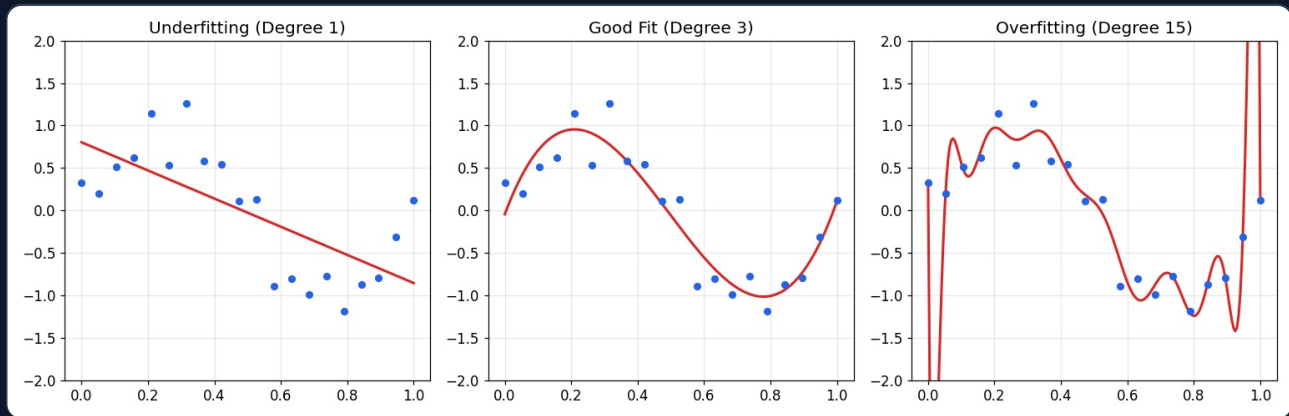
print(f"MAE={mae:.2f}, RMSE={rmse:.2f}, R2={r2:.3f}, Adjusted R2={adj_r2:.3f}")
```

9 Overfitting, Underfitting & Best Fit

Underfitting: The model is too simple and can't learn the pattern in the data (performs poorly on both training and testing).

Best Fit: The model has the right complexity — it learns the general pattern, and performs well on both training and test data.

Overfitting: The model is too complex — it "memorizes" the noise in the training data. It performs very well on training but poorly on new data.



Ridge & Lasso Regression (Regularization)

To control overfitting, we add a **penalty term** to the Cost Function — this is called Regularization.

Ridge Regression (L2 Regularization)

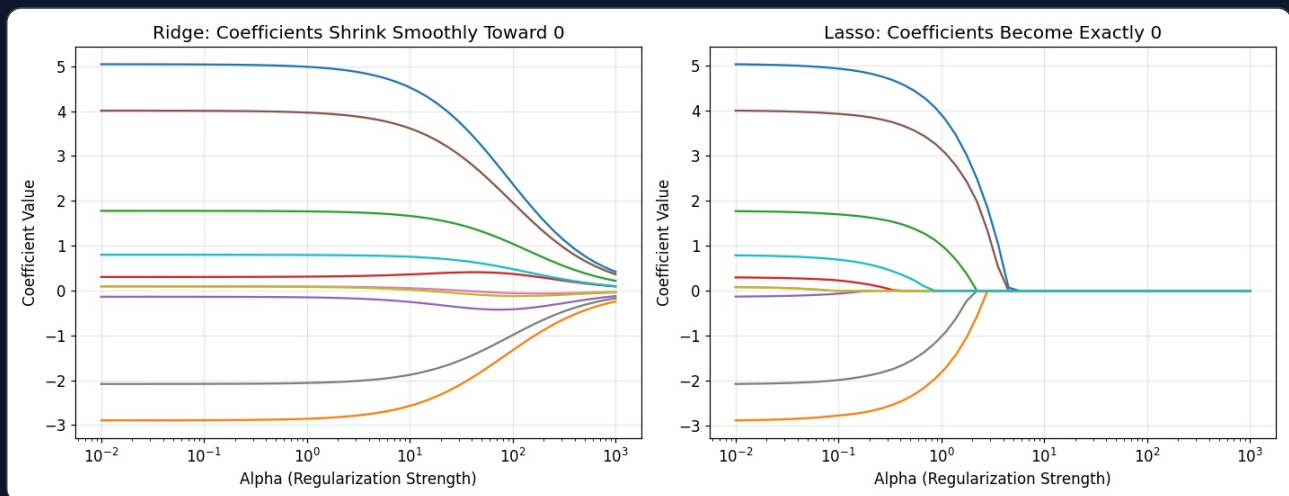
$$\text{Cost} = \text{MSE} + \lambda \cdot \sum m_i^2$$

It shrinks coefficients but never makes them exactly zero. It keeps all features, but reduces their influence.

Lasso Regression (L1 Regularization)

$$\text{Cost} = \text{MSE} + \lambda \cdot \sum |m_i|$$

Lasso can shrink some coefficients all the way to **zero** — meaning it also performs automatic feature selection (removes useless features).



Ridge coefficients shrink smoothly toward zero, Lasso sets them exactly to zero

```
from sklearn.linear_model import Ridge, Lasso
```

```

ridge = Ridge(alpha=1.0) # alpha = lambda (regularization strength)
ridge.fit(X_train, y_train)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

print("Ridge coefficients:", ridge.coef_)
print("Lasso coefficients:", lasso.coef_) # some coefficients will be exactly 0.0

```

Which to use? If you're unsure which features matter and have many features, use **Lasso** (for feature selection). If most features are somewhat useful, **Ridge** works better.

11 StandardScaler (Feature Scaling)

When different features have very different scales (e.g. "size" ranges 100-5000 while "bedrooms" ranges 1-5), the model can end up treating the larger-scale feature as more important — which isn't correct. So we bring all features to the **same scale**.

$$z = (x - \text{mean}) / \text{std_deviation}$$

StandardScaler transforms each feature so its **mean = 0** and **standard deviation = 1**. This makes Gradient Descent faster and more stable, and it's **essential** for Ridge/Lasso (otherwise the penalty affects only large-scale features more).

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit + transform on training data
X_test_scaled = scaler.transform(X_test)      # only transform on test data (never fit again)

```

Important Rule: Only call `fit_transform()` on the training data. On test data, only use `transform()` — otherwise you cause "data leakage" and get incorrect test results.

🔗 Complete End-to-End Example

```

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

# 1. Data
X = np.random.randn(200, 5)
y = 3*X[:,0] - 2*X[:,1] + 5*X[:,2] + np.random.normal(0, 1, 200)

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Scaling

```



```

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 4. Models
models = {
    "Linear": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Lasso": Lasso(alpha=0.1)
}

for name, m in models.items():
    m.fit(X_train_s, y_train)
    pred = m.predict(X_test_s)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)
    print(f"{name}: RMSE={rmse:.3f}, R2={r2:.3f}")

```

Quick Summary

Concept	One-line takeaway
Linear Regression	Fit the best straight line/plane to the data
Residual	Actual – Predicted (error at one point)
MSE / Cost Function	Average squared error — the thing we minimize
Gradient Descent	Step-by-step method to minimize the cost
Learning Rate	Step size — not too small, not too large
Train-Test Split	Check performance on unseen data
R ² / Adjusted R ²	How much variance the model explains (adjusted = penalty for extra features)
Overfitting	Good on training, bad on test (too complex)
Underfitting	Bad on both training and test (too simple)
Ridge (L2)	Shrinks coefficients, doesn't zero them out
Lasso (L1)	Can zero out coefficients (feature selection)
StandardScaler	Brings all features to the same scale (mean=0, std=1)